

# RELATÓRIO TÉCNICO: SISTEMA DE REGISTO DE ESTUDANTES COM TABELA HASH

## 1. INTRODUÇÃO AO PROBLEMA

O gerenciamento eficiente de dados de estudantes é um desafio comum em instituições de ensino. Sistemas tradicionais baseados em arrays ou listas simples podem apresentar limitações quanto à velocidade de busca, especialmente quando o volume de dados cresce.

Este projeto implementa um sistema de registo de estudantes utilizando **tabelas hash**, uma estrutura de dados que permite inserção, busca e remoção rápidas através de uma função de espalhamento. O sistema foi desenvolvido colaborativamente por três programadores (Denilson, Ludovina e Frederico) ao longo de 13 versões (commits), cada um contribuindo com funcionalidades específicas.

O problema principal consiste em:

- Armazenar informações de estudantes (número de matrícula, nome e curso)
- Permitir operações rápidas de inserção, busca e remoção
- Comparar duas estratégias de tratamento de colisões: **encadeamento separado** e **sondagem linear**
- Garantir persistência dos dados em arquivos
- Implementar validações robustas de entrada
- Preservar zeros à esquerda nos números de matrícula

## 2. EXPLICAÇÃO DO ALGORITMO OU ESTRUTURA

### 2.1 Tabela Hash

Uma tabela hash é uma estrutura de dados que associa **chaves de pesquisa** a **valores** através de uma **função de hash**. A função de hash mapeia a chave para um índice dentro de um array, permitindo acesso direto aos dados.

#### Vantagens:

- Inserção, busca e remoção em tempo médio  $O(1)$
- Ideal para conjuntos de dados onde a busca rápida é prioritária

#### Desvantagens:

- Necessidade de tratar colisões (quando duas chaves diferentes geram o mesmo índice)
- Performance degrada com fator de carga elevado

### 2.2 Função de Hash Implementada

c

```
int funcaoHash(int numero) {  
  
    return numero % TAM; // TAM = 10  
  
}
```

É uma função simples baseada no módulo (resto da divisão). O índice é calculado como o resto da divisão do número do estudante pelo tamanho da tabela (10).

### 2.3 Tratamento de Colisões

O sistema implementa **duas técnicas** de tratamento de colisões para comparação:

#### a) Encadeamento Separado

- Cada posição da tabela contém um ponteiro para uma lista ligada
- Quando ocorre colisão, o novo elemento é adicionado ao final da lista

- **Vantagem:** Não há limite de elementos por índice
- **Desvantagem:** Necessidade de alocação dinâmica de memória

## b) Sondagem Linear

- Quando ocorre colisão, procura-se a próxima posição livre (índice + 1, +2, etc.)
- **Vantagem:** Simples de implementar, memória contígua
- **Desvantagem:** Agrupamento primário (clustering) - tendência a formar blocos ocupados

## 2.4 Persistência em Arquivo

Os dados são salvos em dois arquivos texto:

- estudantes\_encadeamento.txt: dados da tabela com encadeamento
- estudantes\_linear.txt: dados da tabela com sondagem linear

Formato: índice:numero\_str:numero\_int:nome:curso

## 3. IMPLEMENTAÇÃO EM C

### 3.1 Estruturas de Dados

c

```
#define TAM 10
```

```
#define MAX_NOME 50
```

```
#define MAX_CURSO 50
```

```
#define MAX_STR 20
```

```
typedef struct Estudante {
```

```
    char numeroStr[MAX_STR]; // Número como string (preserva zeros)
```

```
    int numero; // Número para cálculo do hash
```

```
    char nome[MAX_NOME];
```

```

char curso[MAX_CURSO];

struct Estudante* prox;    // Ponteiro para encadeamento
} Estudante;

// Tabelas

Estudante* tabelaEncadeamento[TAM]; // Array de ponteiros

Estudante tabelaLinear[TAM];        // Array de estruturas

```

### 3.2 Funções Principais

#### Inicialização

```

c

void inicializarTabelas() {

    for(int i = 0; i < TAM; i++) {

        tabelaEncadeamento[i] = NULL;

        tabelaLinear[i].numero = -1; // -1 indica posição vazia

        strcpy(tabelaLinear[i].numeroStr, "");

    }

}

```

#### Inserção com Encadeamento

```

c

void inserirEncadeamento(char numeroStr[], int numero, char nome[], char curso[]) {

    int indice = funcaoHash(numero);

    Estudante* novo = (Estudante*)malloc(sizeof(Estudante));

```

```
strcpy(novo->numeroStr, numeroStr);
```

```
novo->numero = numero;
```

```
strcpy(novo->nome, nome);
```

```
strcpy(novo->curso, curso);
```

```
novo->prox = NULL;
```

```
if(tabelaEncadeamento[indice] == NULL) {
```

```
    tabelaEncadeamento[indice] = novo;
```

```
} else {
```

```
    Estudante* temp = tabelaEncadeamento[indice];
```

```
    while(temp->prox != NULL) temp = temp->prox;
```

```
    temp->prox = novo;
```

```
}
```

```
}
```

### **Inserção com Sondagem Linear**

c

```
void inserirLinear(char numeroStr[], int numero, char nome[], char curso[]) {
```

```
    int indice = funcaoHash(numero);
```

```
    int original = indice;
```

```
    while(tabelaLinear[indice].numero != -1) {
```

```
        indice = (indice + 1) % TAM; // Sondagem linear circular
```

```
        if(indice == original) {
```

```

        printf("ERRO: Tabela linear cheia!\n");

        return;

    }

}

```

```

strcpy(tabelaLinear[indice].numeroStr, numeroStr);

tabelaLinear[indice].numero = numero;

strcpy(tabelaLinear[indice].nome, nome);

strcpy(tabelaLinear[indice].curso, curso);

}

```

## Busca

c

```

void procurarEncadeamento() {

    char numeroStr[MAX_STR];

    printf("Digite o numero do estudante: ");

    fgets(numeroStr, MAX_STR, stdin);

    numeroStr[strcspn(numeroStr, "\n")] = 0;


    int numero = atoi(numeroStr);

    int indice = funcaoHash(numero);

    Estudante* temp = tabelaEncadeamento[indice];

    int encontrado = 0;

```

```

while(temp != NULL) {
    if(temp->numero == numero) {
        printf("\nESTUDANTE ENCONTRADO (Encadeamento)\n");
        printf("Numero: %s\n", temp->numeroStr);
        printf("Nome: %s\n", temp->nome);
        printf("Curso: %s\n", temp->curso);
        encontrado = 1;
        break;
    }
    temp = temp->prox;
}

if(!encontrado) printf("Estudante nao encontrado.\n");
}

```

### **Remoção Sincronizada**

c

```

void removerDeAmbasTabelas(char numeroStr[]) {
    removerDaTabelaEncadeamento(numeroStr);
    removerDaTabelaLinear(numeroStr);
    printf("Estudante removido de ambas as tabelas.\n");
}

```

### **4.1 Funcionalidades Implementadas**

O sistema final oferece:

| Funcionalidade       | Descrição                                 | Commit    |
|----------------------|-------------------------------------------|-----------|
| Inicialização        | Setup das tabelas hash                    | Commit 1  |
| Validações           | Verificação de campos vazios e caracteres | Commit 2  |
| Inserção             | Cadastro de novos estudantes              | Commit 3  |
| Encadeamento         | Tratamento de colisões com listas         | Commit 4  |
| Sondagem Linear      | Tratamento de colisões sequencial         | Commit 5  |
| Exibição             | Visualização de ambas as tabelas          | Commit 6  |
| Busca                | Procura por número de estudante           | Commit 7  |
| Persistência         | Salvamento em arquivos                    | Commit 8  |
| Remoção              | Exclusão de registros                     | Commit 9  |
| Sincronização        | Remoção simultânea nas duas tabelas       | Commit 10 |
| Normalização         | Conversão para minúsculas                 | Commit 11 |
| Validação de espaços | Prevenção de nomes inválidos              | Commit 12 |
| Correção de buffer   | Preservação de zeros à esquerda           | Commit 13 |

## 4.2 Exemplo de Execução

text

--- INSERIR ESTUDANTE ---

Numero do estudante (apenas digitos): 001

Numero validado: 001

Nome do estudante: Joao Silva

Curso do estudante: Engenharia Informatica

[ENCADEAMENTO] Numero: 001 (hash: 1)

Indice calculado: 1

Indice 1: vazio - inserido com sucesso!

[SONDAGEM LINEAR] Numero: 001 (hash: 1)

Indice calculado: 1

Indice 1: vazio - inserido com sucesso!

### 4.3 Testes Realizados

| Teste             | Cenário                    | Resultado                             |
|-------------------|----------------------------|---------------------------------------|
| Inserção normal   | Número 101 → índice 1      | Sucesso                               |
| Colisão           | Número 201 também índice 1 | Encadeamento: lista; Linear: índice 2 |
| Busca existente   | Número 001                 | Encontrado em ambas                   |
| Busca inexistente | Número 999                 | Não encontrado                        |
| Remoção           | Número 101                 | Removido de ambas                     |
| Persistência      | Após reiniciar programa    | Dados recuperados                     |
| Zeros à esquerda  | "001" vs 1                 | Preservado como string                |

## 5. ANÁLISE DE COMPLEXIDADE

### 5.1 Complexidade Temporal

| Operação        | Encadeamento                   | Sondagem Linear                |
|-----------------|--------------------------------|--------------------------------|
| <b>Inserção</b> | $O(1)$ médio, $O(n)$ pior caso | $O(1)$ médio, $O(n)$ pior caso |
| <b>Busca</b>    | $O(1)$ médio, $O(n)$ pior caso | $O(1)$ médio, $O(n)$ pior caso |
| <b>Remoção</b>  | $O(1)$ médio, $O(n)$ pior caso | $O(1)$ médio, $O(n)$ pior caso |

Onde:

- $n$  = número de elementos na tabela
- TAM = 10 (constante)

### 5.2 Complexidade Espacial

- **Encadeamento:**  $O(n)$  + overhead dos ponteiros
- **Sondagem Linear:**  $O(\text{TAM})$  fixo, independente de  $n$

### 5.3 Fator de Carga

Fator de carga =  $n / \text{TAM}$

- Ideal: manter fator de carga  $\leq 0.7$
- Quando TAM = 10 e fator de carga = 0.7  $\rightarrow$  máximo 7 estudantes